

TP0 : Prise en main de l'outil Modelio

1. Présentation et téléchargement de l'outil Modelio

Modelio est un outil de modélisation UML disponible en version entreprise et community. Toutes les versions open source (la version utilisée pour la réalisation des exercices de ce TP est: 4.1) sont téléchargeables à partir du site de référence: <https://www.modelio.org/>. Modelio supporte la plupart des diagrammes spécifiés par UML 2.0. Des générateurs de code (Java, C++, C#) y sont par exemple disponibles et d'autres modules peuvent être par la suite rajoutés. Vous pouvez retrouver la liste des fonctionnalités de Modelio Open Source dans <https://www.modelio.org/about-modelio/features.html>.

Nous utiliserons dans ce TP la version (4.1 .0) de Modelio. Pour cela, télécharger la version correspondante à votre système d'exploitation à partir de l'onglet 'downloads' de la page <https://www.modelio.org/downloads/download-modelio.html>. (Onglet Previous versions) Sinon, la version adéquate au système Windows vous sera fournie par votre assistant de TP.

2. Installation

Modelio 4.1.0 est une application java qui est délivré avec un installable pour (Windows et linux). Par contre, pour MacOS X, il suffit simplement de décompresser le dossier téléchargé et de lancer l'application. Par ailleurs, l'application nécessite la dernière version de Java JDK

8. En cas de dysfonctionnement ou de problèmes pour faire fonctionner modelio, pensez à mettre à jour votre JDK.

3. Préparation du workspace

Le workspace est le répertoire de travail dans lequel seront stockés tous vos projets. Vous pourrez définir dans votre workspace un nouveau projet par TP ou par étude de cas.

1. Par défaut, Modelio utilise un workspace dans le dossier ~/modelio
2. Pour changer de workspace, File > Switch Workspace par le menu de Modelio.
3. Créer, dans un premier temps, un dossier (pour votre workspace) nommé TP_SGL pour abriter tous vos TP.

4. Création d'un projet

- Pour créer un projet, **File > Create a project** dans le menu. Pour ce TP, nommez votre projet TPPrise_en_main. Un package nommé TPPrise_en_main est automatiquement créé.

Avant de valider votre projet, cochez la case Java Project.

- Pour créer un nouveau projet, il faut fermer le projet ouvert, **File>Close Project**.
- Pour chaque diagramme explorer, déposer et voir les éléments constitutifs.

5. Générez automatiquement une classe dans Modelio à partir du code source

1. Créer un fichier « Adresse.java » dans le woks pace TPPrise_en_main, puis saisissez la classe suivante.

```
class Adresse {  
    private int numero ;  
    private String rue ;  
    private String codePostale ;  
    private String ville ;  
}
```

2. Pour faire apparaître la classe adresse dans modelio, vous cliquez droit sur le paquage TPPrise_en_main, puis sélectionner Java Designer > Reverse en cherchant à importer des codes sources en .java et pas des binaires (.class). Sélectionnez le dossier dans lequel se trouve le fichier « Adresse.java ». Cliquez sur Suivant jusqu'à pouvoir faire Reverse.
3. La classe Adresse apparaîtra dans l'arborescence mais pas encore dans le diagramme. Si vous souhaitez faire apparaître la classe dans le diagramme, glissez là de l'arborescence vers le diagramme.
4. Pour faire apparaître les attributs, glissez-les de l'arborescence vers le diagramme.

6. Générer la documentation de vos modèles

- Pour générer la documentation de vos modèles, rajouter le module **WebModelPublisher** a votre projet, en cliquant sur : **configuration>module>add** et choisissez le module **WebModelPublisher**, puis cliquer sur **deploy in the projet**.

Ensuite, Cliquez sur le bouton droit du paquetage **tp_prise_en main> web Model Publisher> generate**.

Travail supplémentaire :

1. Générer le diagramme correspondant au code JAVA suivant.

```
public class Figure {  
    protected String couleur;  
    protected String getCouleur ( ) {  
        return couleur; }  
    protected void setCouleur ( String c ) {  
        couleur = c; } }  
  
public class Point {  
    private float x;  
    private float y;  
    public float getX ( ) {  
        return x; }  
    public float getY ( ) {  
        return y; }  
    public void Point ( float x, float y) { }  
}  
public class Cercle extends Figure {  
    private float rayon;  
    private Point centre; }  
}
```